Lattice IRRuntime

Feature Set Reference

Lattice Project

March 2026

Contents

1 Overview **3**

2 Input Formats **3**

3 TextIR Syntax **3**
3.1 Module header . . . 3
3.2 Type declaration keywords . . . 3
3.3 Member declarations . . . 4

4 Type System **4**
4.1 Primitive types . . . 4
4.2 User-defined types . . . 5

5 Instruction Set **5**
5.1 Stack manipulation . . . 5
5.2 Load constants . . . 5
5.3 Variables and arguments . . . 5
5.4 Fields . . . 6
5.5 Arithmetic . . . 6
5.6 Logical . . . 6
5.7 Comparison . . . 6
5.8 Objects and arrays . . . 7
5.9 Type conversion and testing . . . 7
5.10 Method calls . . . 7
5.11 Control flow . . . 8

6 Runtime API **8**
6.1 Construction . . . 8
6.2 Running a program . . . 8
6.3 Calling individual methods . . . 8
6.4 Registering native methods . . . 9
6.5 Observability hooks . . . 9
6.6 Accessing the call stack . . . 9

7 CLR Type Registration **9**

8 Built-in Native Methods **10**

9 Value Coercion Rules **11**

10 Exception Model **11**

11 Attributes **11**

12 Limitations and Known Gaps **11**

1 Overview

Lattice is a stack-based virtual machine written in C# (.NET 10). It executes programs described in an *Intermediate Representation* (IR). The runtime loads an IR module, resolves types and methods, and executes instructions on a per-frame evaluation stack.

Key design goals:

- Simple, human-readable text IR format (`TextIR`).
- Seamless interop with native C# types and methods.
- Embeddable — host applications can load a module, call IR methods, and register C# types or callbacks without writing a parser.

2 Input Formats

The runtime accepts three equivalent input representations, selected via the `InputFormat` enum or auto-detected.

Format	Enum value	Description
`Auto`	`InputFormat.Auto`	JSON if starts with { or [, else TextIR
`TextIr`	`InputFormat.TextIr`	Human-readable text IR (see Section 3)
`Json`	`InputFormat.Json`	Serialised `ModuleDto` JSON
`Ast`	`InputFormat.Ast`	`ModuleNode` AST lowered by `AstLowering`

3 TextIR Syntax

A TextIR file begins with a module header, followed by one or more type definitions.

Listing 1: Minimal TextIR module

```
module MyApp version 1.0.0

class Program {
    static method Main() -> void {
        ldstr "Hello, World!"
        call System.Console.WriteLine(System.string) -> void
        ret
    }
}
```

3.1 Module header

```
module <name> version <major>.<minor>[.<patch>]
```

3.2 Type declaration keywords

Keyword	Meaning
`class`	Reference type
`interface`	Interface (pure virtual contract)
`struct`	Value type
`enum`	Enumeration type

Type modifiers (`public`, `private`, `abstract`, `sealed`) may precede the keyword. Inheritance is expressed with `::`

```
class Animal {
    field name: string
    method Speak() -> void { ... }
}

class Dog : Animal {
    override method Speak() -> void implements Animal.Speak { ... }
}
```

3.3 Member declarations

```
// Field
[static] field <name>: <type>

// Method
[static] [virtual] [override] [abstract] method <name>(<params>) -> <rettype> [implements <Interface>.<MethodName>] { ... }

// Constructor
constructor(<params>) { ... }

// Local variable
local <name>: <type>
```

4 Type System

4.1 Primitive types

IR name(s)	Alias(es)	CLR type
void	system.void	void
bool	system.boolean	bool
int8	system.sbyte	sbyte
uint8	system.byte	byte
int16	system.int16	short
uint16	system.uint16	ushort
int32	system.int32	int
uint32	system.uint32	uint
int64	system.int64	long
uint64	system.uint64	ulong
float32, single	system.single	float
float64, double	system.double	double
char	system.char	char
string	system.string	string
object	system.object	object
decimal	system.decimal	decimal
datetime	system.datetime	DateTime
timespan	system.timespan	TimeSpan
guid	system.guid	Guid

Type names are case-insensitive in IR operands. The runtime normalises them to lower-case before lookup, so `System.String`, `system.string`, and `string` are all equivalent.

4.2 User-defined types

User-defined types are declared in the module and managed by the VM as `ManagedObject` instances at runtime. Each instance carries:

- A type name string (`TypeName`).
- A dictionary of named field values.
- Optional CLR-backing object (see Section 7).
- Attached attribute instances.
- Arbitrary metadata slots.

5 Instruction Set

5.1 Stack manipulation

Opcode	Description
`nop`	No operation.
`dup`	Duplicate the top value on the evaluation stack.
`pop`	Discard the top value.
`ldnull`	Push `null` onto the stack.

5.2 Load constants

Opcode	Description
`ldc`	Push a typed constant. Operand: {`value, type`}. Type defaults to `int64`.
`ldc.i4`	Push a 32-bit integer constant.
`ldc.i8`	Push a 64-bit integer constant.
`ldc.r4`	Push a 32-bit float constant.
`ldc.r8`	Push a 64-bit float constant.
`ldstr`	Push a string constant. Operand: {`value`}.

5.3 Variables and arguments

Opcode	Description
`ldarg`	Push a method argument by index or name (including `this`).
`starg`	Pop and store into a named argument slot.
`ldloc`	Push a local variable by name.
`stloc`	Pop and store into a local variable by name.

5.4 Fields

Opcode	Description
`ldfld`	Pop an object from the stack (or use `this`) and push the named instance field.
`stfld`	Pop a value, then pop an object; store the value into the named instance field.
`ldsfld`	Push a static field. Operand: {`field`: {`declaringType, name`}}.
`stsfld`	Pop a value; store it into a static field.

When the object wraps a CLR-backed instance (see Section 7), `ldfld`/`stfld` use reflection to read/write the corresponding CLR property or field.

5.5 Arithmetic

Opcode	Description
`add`	Pop b, pop a, push $a + b$.
`sub`	Push $a - b$.
`mul`	Push $a \times b$.
`div`	Push $a \div b$.
`rem`	Push a mod b.
`neg`	Pop v, push $-v$.

If either operand is a floating-point value, both are promoted to `double`. Integer operations return `long`.

5.6 Logical

Opcode	Description
`not`	Pop v, push `!ToBool(`v`)`.

5.7 Comparison

Each opcode pops b then a and pushes a `bool`.

Opcode	Condition
`ceq`	$a = b$
`cne`	$a \neq b$
`cgt`	$a > b$
`cge`	$a \geq b$
`clt`	$a < b$
`cle`	$a \leq b$

If either operand is a `string` the comparison is performed lexicographically (ordinal). Boolean operands are supported for `ceq`/`cne`. All other operands are promoted to `double`.

5.8 Objects and arrays

Opcode	Description
`newobj`	Create a new `ManagedObject` of the given type. If the type is registered as a CLR type, a real CLR instance is created and attached.
`newarr`	Push a new `List<object?>` (dynamically-typed array).
`ldelem`	Pop index, pop array, push `array[index]`.
`stelem`	Pop value, pop index, pop array; set `array[index] = value`.

5.9 Type conversion and testing

Opcode	Description
`conv`	Pop a value and push it converted to the named target type.
`castclass`	Assert that the top-of-stack `ManagedObject` has the expected type name; throws `InvalidCastException` otherwise.
`isinst`	Pop a value; push `true` if it is a `ManagedObject` with the given type name, otherwise `false`.

5.10 Method calls

Opcode	Description
`call`	Static or instance call. Operand: {`method: {declaringType, name, returnType, parameterTypes[]}`}.
`callvirt`	Virtual call. Identical to `call` but additionally pops an object instance from the stack.
`ret`	Return from the current frame. If a value is on the stack it is returned to the caller.

Call resolution order

1. Exact signature lookup in the native method dictionary.
2. Simplified-signature lookup (parameter types reduced to unqualified lower-case short names, e.g. `System.String` → `string`).
3. Reflection-based CLR invocation (only when `enableReflectionNativeMethods` is `true`).
4. IR method lookup in the loaded module.

5.11 Control flow

Opcode	Description
`if`	Pop (or evaluate) a condition; execute `thenBlock` or optional `elseBlock`.
`while`	Repeatedly evaluate condition and execute `body`. Supports `break`/`continue`.
`break`	Exit the nearest enclosing `while` loop.
`continue`	Skip to the next iteration of the nearest `while` loop.
`try`	Structured exception handling with `tryBlock`, zero or more `catchBlocks`, and optional `finallyBlock`.
`throw`	Pop a value (VM exception object or CLR exception) and set it as a pending exception on the current frame.

Condition kinds A condition embedded in `if` or `while` is one of:

Kind	Evaluation
`stack`	Pop the top-of-stack value and coerce to `bool`.
`binary`	Pop r, pop l; compare using the specified `operation` opcode.
`expression`	Execute a single embedded `InstructionDto` and pop the result.
`block`	Execute a sequence of instructions and pop the final result.

6 Runtime API

6.1 Construction

```
// Load from a TextIR/JSON/AST string at construction time
var rt = new IRRuntime(irSource);

// Construct empty, load later
var rt = new IRRuntime();
rt.LoadModule(irSource, InputFormat.TextIr);

// Enable reflection-based CLR interop
var rt = new IRRuntime(irSource, enableReflectionNativeMethods: true);
```

6.2 Running a program

```
// Requires a static Program.Main() method in the module
rt.Run();
```

6.3 Calling individual methods

```
// Untyped call
object? result = rt.CallMethod("MyType.MyMethod", arg1, arg2);

// Generic typed call with auto-conversion
int n = rt.CallMethod<int>("Calculator.Add", 3, 4);
```

6.4 Registering native methods

Bind a C# delegate to an IR call signature:

```
rt.RegisterNativeMethod(
    "MathHelper.Sqrt(float64)",
    (instance, args) => Math.Sqrt((double)args[0]!));
```

The signature format is:

`DeclaringType.MethodName(param1Type,param2Type,...)`

Both fully-qualified (`System.String`) and short (`string`) parameter type names are tried automatically.

6.5 Observability hooks

```
// Called before every instruction (debugger / profiler hook)
rt.OnStep = (frame, instr) => Console.WriteLine(instr.opCode);

// Called when an unhandled exception escapes
rt.OnException = ex => Console.Error.WriteLine(ex);
```

6.6 Accessing the call stack

```
IReadOnlyList<CallFrame> stack = rt.CallStack;
```

7 CLR Type Registration

Host code can register C# classes so the VM can instantiate, field-access, and call methods on them as if they were IR-defined types.

```
// Register by type parameter (uses FullName as IR name by default)
IRRuntime.RegisterClrType<MyService>();

// Register with a custom IR name
IRRuntime.RegisterClrType<MyService>("Services.MyService");

// Register via a Type object
IRRuntime.RegisterClrType("Services.MyService", typeof(MyService));
```

Once registered:

- `newobj` creates a real CLR instance via `Activator.CreateInstance` and attaches it to the `ManagedObject` wrapper.
- `ldfld/stfld` use reflection to read/write properties and fields on the CLR instance, falling back to the VM field dictionary if the member is not found.
- `ldsfld/stsfld` reflectively access static CLR fields/properties.
- **Reflected calls** (`EnableReflectionNativeMethods`) receive the real CLR instance, not the `ManagedObject` wrapper.
- The type name is also consulted by `ResolveClrType`, so it can be used in cast expressions and catch clauses.

Example: Registering a custom service

```
public class Counter {
    public int Value { get; set; }
    public void Increment() => Value++;
    public int GetValue() => Value;
}

IRRuntime.RegisterClrType<Counter>("Counter");

var rt = new IRRuntime("""
    module CounterDemo version 1.0

    class Program {
        static method Main() -> void {
            newobj Counter
            callvirt Counter.Increment()
            callvirt Counter.GetValue() -> int32
            call System.Console.WriteLine(int32) -> void
            ret
        }
    }
""", enableReflectionNativeMethods: true);

rt.Run(); // prints: 1
```

8 Built-in Native Methods

The following native methods are pre-registered on every `IRRuntime` instance.

Signature	Behaviour
`System.Console.WriteLine(string)`	`Console.WriteLine(args[0])`
`System.Console.WriteLine(int32)`	`Console.WriteLine(args[0])`
`System.Console.WriteLine(object)`	`Console.WriteLine(args[0])`
`System.Console.WriteLine(string,string)`	Concatenates all args and writes a line
`System.Console.WriteLine(string,object)`	`string.Format` then writes a line
`System.Console.Write(string)`	`Console.Write(args[0])`
`System.Console.Write(string,string)`	Concatenates all args and writes
`System.Console.ReadLine()`	Returns `Console.ReadLine()`
`System.Console.Clear()`	`Console.Clear()`
`System.Console.Beep()`	`Console.Beep()`
`System.String.Concat(string,string)`	`string.Concat(args[0], args[1])`
`System.String.Format(string,object)`	`string.Format(fmt, args[1])`
`System.String.Format(string,object,object)`	`string.Format(fmt, a, b)`
`System.String.IsNullOrEmpty(string)`	`string.IsNullOrEmpty(args[0])`
`System.Object.ToString()`	`instance?.ToString()`
`System.Object.GetType()`	`instance?.GetType()`

9 Value Coercion Rules

Target	Conversion
`bool`	`null`/`0`/`""` → `false`; everything else → `true`.
`long`	Numeric types truncated/cast; strings parsed with `InvariantCulture`.
`double`	Same as `long` but without truncation.
`string`	`object.ToString()`, or `null` if value is `null`.
CLR type	`Convert.ChangeType` fallback after the typed fast-paths.

10 Exception Model

- IR code raises exceptions with `throw`, which pops a value (any object, including a `ManagedObject`) and stores it as `frame.PendingException`.
- `try`/`catch`/`finally` blocks are first-class instructions handled inline by the interpreter.
- Catch clauses match by:
  1. Catch-all (no type specified).
  2. IR `ManagedObject` type name match.
  3. CLR `IsAssignableFrom` check for CLR exception types.
- If no catch clause matches, the pending exception propagates up the call stack. At the top level it is delivered to `OnException`.

11 Attributes

Custom attributes can be applied to IR types, fields, and methods using the standard `@AttributeName(args)` syntax in the TextIR. At runtime, attribute instances are created as `ManagedObject`s and attached to the corresponding `ManagedObject` via `AttachAttributesFromIR`. Host code can retrieve them with:

```
ManagedObject instance = ...;
var attr = instance.GetAttribute<MyManagedAttribute>();
```

12 Limitations and Known Gaps

- Generics and generic type parameters are not yet supported.
- Arrays are implemented as `List<object?>`; typed arrays are not distinguished at runtime.
- `castclass` and `isinst` only match by exact `TypeName` string; inheritance-aware checks are not performed for IR types.
- `newobj` uses `Activator.CreateInstance()`; constructor arguments for CLR-backed types must be passed via native method wrappers or IR constructors.
- The `while (stack)` condition idiom requires the caller to push a fresh truthy value before each iteration; an exhausted stack is treated as `false` (loop exits gracefully).